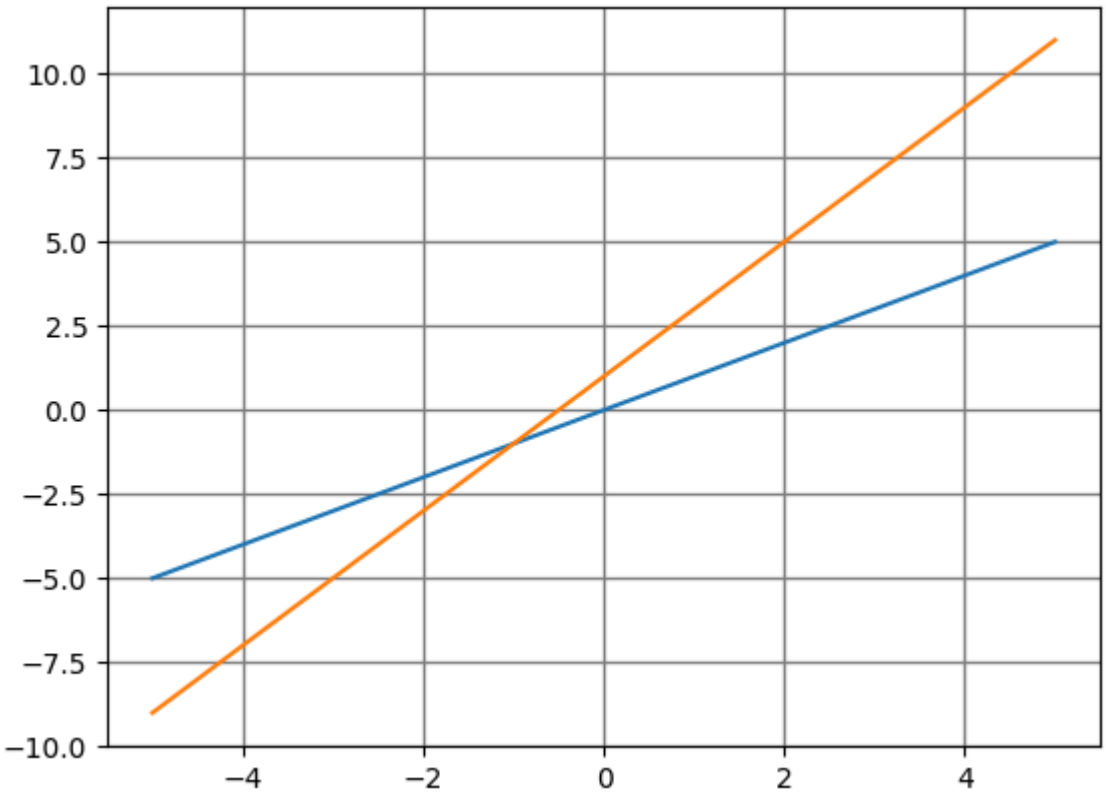
In [1]:
```
import numpy as np
import matplotlib.pyplot as plt
def plot(*funcs, x_min=-5, x_max=5, marker='', step=None):
    plt.grid(color='gray', linestyle='-', linewidth=1)
    xs = np.arange(x_min, x_max, step) if step else np.linspace(x_min, x_max, 50
    for f in funcs:
        plt.plot(xs, np.vectorize(f)(xs), marker)
```

In [2]:
```
plot(lambda x: x, lambda x: 2*x+1)
```



In [3]:
```
%%latex
$a$ Geld des Ersten.
$b$ Geld des Zweiten.
$c$ Geld des Dritten.
$z$ ist die Zeche.
\begin{align}
a + \frac{1}{3}b &= z \\
b + \frac{1}{2}a &= z \\
c + \frac{3}{4}a &= z \\
a + b + c &= 45 \\
\\
a + \frac{1}{3}b &= b + \frac{1}{2}a | - \frac{1}{2}a - \frac{1}{3}b \\
\frac{1}{2}a &= \frac{2}{3}b | *\frac{3}{2} \\
\frac{3}{4}a &= b \\
\\
a + \frac{1}{3}b &= c + \frac{3}{4}a \\
a + \frac{1}{3}\frac{3}{4}a &= c + \frac{3}{4}a \\
a + \frac{1}{4}a &= c + \frac{3}{4}a | - \frac{3}{4}a \\
\frac{2}{4}a &= c \\
\\
a + b + c &= 45 \\
a + \frac{3}{4}a + \frac{2}{4}a &= 45 \\
\frac{9}{4}a &= 45|*\frac{4}{9} \\
```

```
a &= 20 \\
\\
\frac{3}{4}a &= b \\
\frac{3}{4}20 &= b \\
15 &= b \\
\\
\frac{2}{4}a &= c \\
\frac{2}{4}20 &= c \\
10 &= c \\
\\
a + \frac{1}{3}b = 20 + \frac{1}{3}*15 = 25
\end{align}
```

a Geld des Ersten. b Geld des Zweiten. c Geld des Dritten. z ist die Zeche.

$$a + \frac{1}{3}b = z \tag{1}$$
$$b + \frac{1}{2}a = z \tag{2}$$
$$c + \frac{3}{4}a = z \tag{3}$$
$$a + b + c = 45 \tag{4}$$
$$\tag{5}$$
$$a + \frac{1}{3}b = b + \frac{1}{2}a| - \frac{1}{2}a - \frac{1}{3}b \tag{6}$$
$$\frac{1}{2}a = \frac{2}{3}b| * \frac{3}{2} \tag{7}$$
$$\frac{3}{4}a = b \tag{8}$$
$$\tag{9}$$
$$a + \frac{1}{3}b = c + \frac{3}{4}a \tag{10}$$
$$a + \frac{1}{3}\frac{3}{4}a = c + \frac{3}{4}a \tag{11}$$
$$a + \frac{1}{4}a = c + \frac{3}{4}a| - \frac{3}{4}a \tag{12}$$
$$\frac{2}{4}a = c \tag{13}$$
$$\tag{14}$$
$$a + b + c = 45 \tag{15}$$
$$a + \frac{3}{4}a + \frac{2}{4}a = 45 \tag{16}$$
$$\frac{9}{4}a = 45| * \frac{4}{9} \tag{17}$$
$$a = 20 \tag{18}$$
$$\tag{19}$$
$$\frac{3}{4}a = b \tag{20}$$
$$\frac{3}{4}20 = b \tag{21}$$
$$15 = b \tag{22}$$
$$\tag{23}$$
$$\frac{2}{4}a = c \tag{24}$$
$$\frac{2}{4}20 = c \tag{25}$$
$$10 = c \tag{26}$$
$$\tag{27}$$
$$a + \frac{1}{3}b = 20 + \frac{1}{3} * 15 = 25 \tag{28}$$

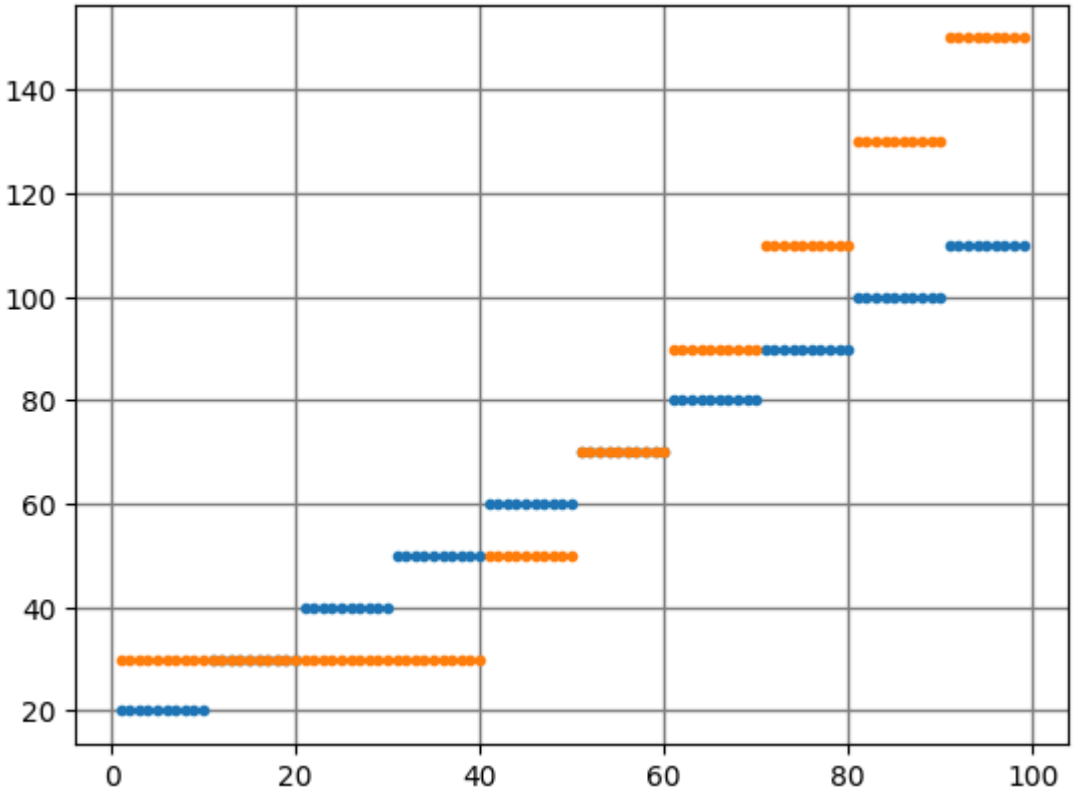
In [4]:
```
def a(x):
    # Aufrunden auf nächste Zehnerstelle
    x = (x + 9) // 10 * 10
    return 10 + x / 10 * 10


def b(x):
    if x <= 40:
        return 30
```

```
    else:
        weitere = x - 40
        # Aufrunden auf nächste Zehnerstelle
        weitere = (weitere + 9) // 10 * 10
        return 30 + weitere / 10 * 20


def best(x):
    return min(a(x), b(x))

plot(a, b, #best,
     x_min=1, x_max=100, marker='.', step=1)
```



In [5]:
```
%%latex
$3*(x-5y ) = 3a$
```

$3 * (x - 5y) = 3a$

In [6]:
```
%%latex
$3x - 15y = 3a$
```

$3x - 15y = 3a$

In [7]:
```
%%latex
$-15=b$
```

$-15 = b$

In [8]:
```
%%latex
$a = 57/3 = 19$
```

$a = 57/3 = 19$

In [9]:
```
%%latex
\begin{align}
c*(x -5y) &= ca | -ca\\
cx -5cy-ca&=0 \\
\end{align}
```

$$c*(x-5y) = ca| - ca \tag{29}$$
$$cx - 5cy - ca = 0 \tag{30}$$

In [10]:
```
%%latex
\begin{align}
3x + by &= 57 | -57\\
3x + by - 57&=0 \\
\end{align}
```

$$3x + by = 57| - 57 \tag{31}$$
$$3x + by - 57 = 0 \tag{32}$$

In [11]:
```
%%latex
\begin{align}
3x + by - 57=cx -5cy-ca \\
\end{align}
```

$$3x + by - 57 = cx - 5cy - ca \tag{33}$$

In [12]:
```
%%latex
$xy = 3 + 2x | /x$
```

$xy = 3 + 2x|/x$

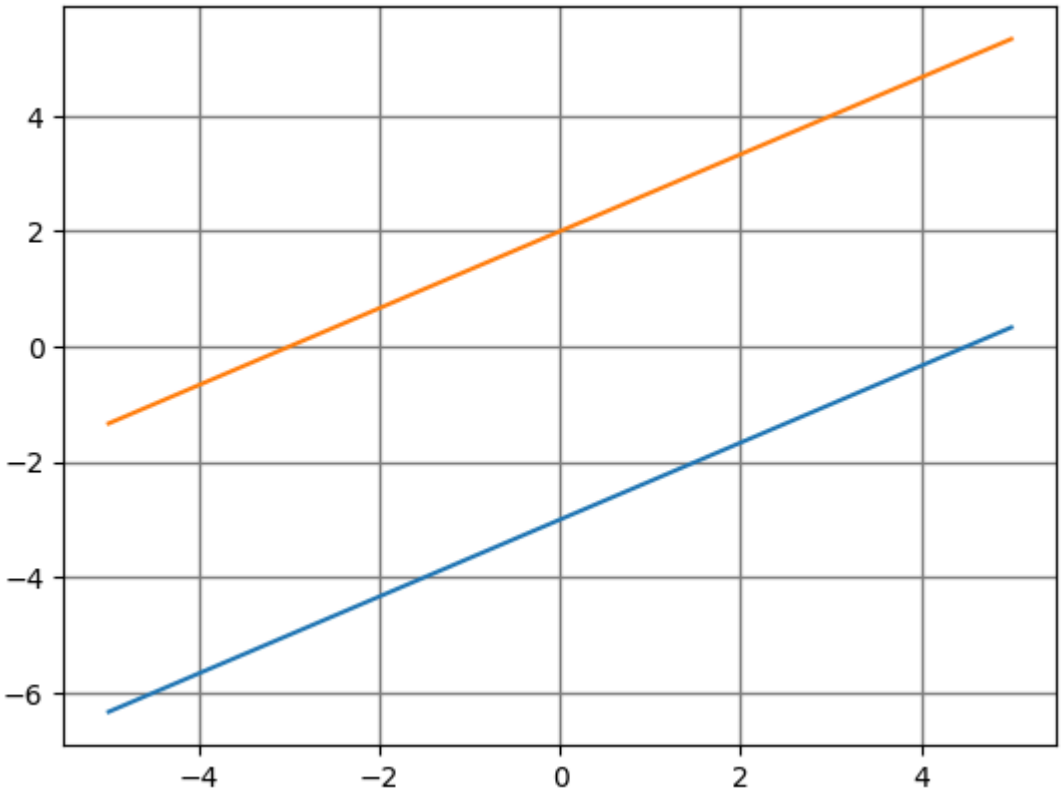
In [13]:
```
%%latex
$y = 3 / x + 2$
```

$y = 3/x + 2$

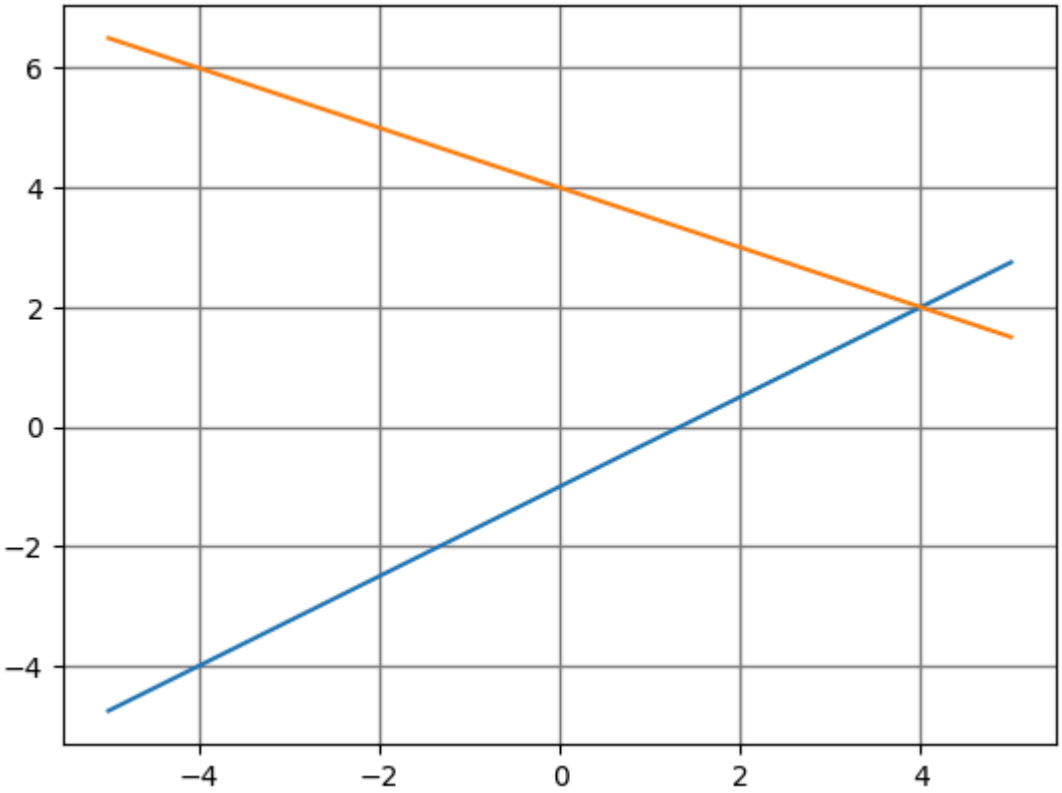
In [14]:
```
%%latex
$x^1 = x^{2+(-1)} = x^2 * x ^{-1} = x * x * x^{-1}$
```

$x^1 = x^{2+(-1)} = x^2 * x^{-1} = x * x * x^{-1}$

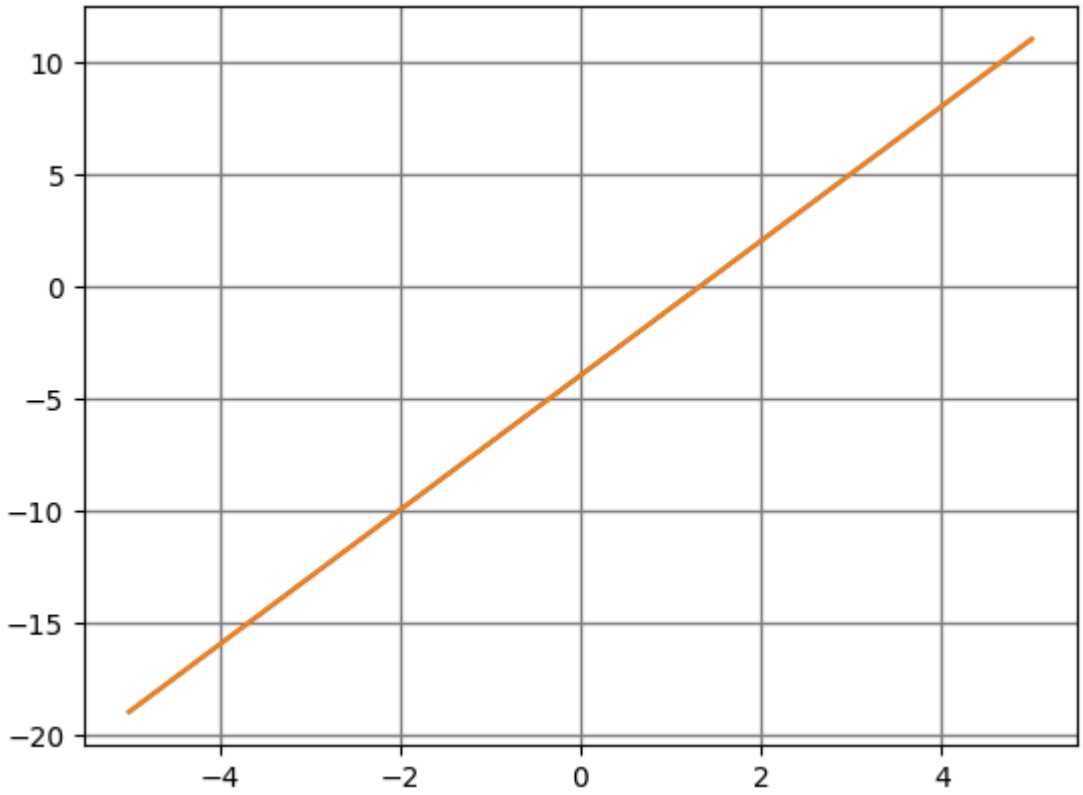
In [15]:
```
plot(lambda x:2/3*x-3, lambda x:2/3*x+2)
```



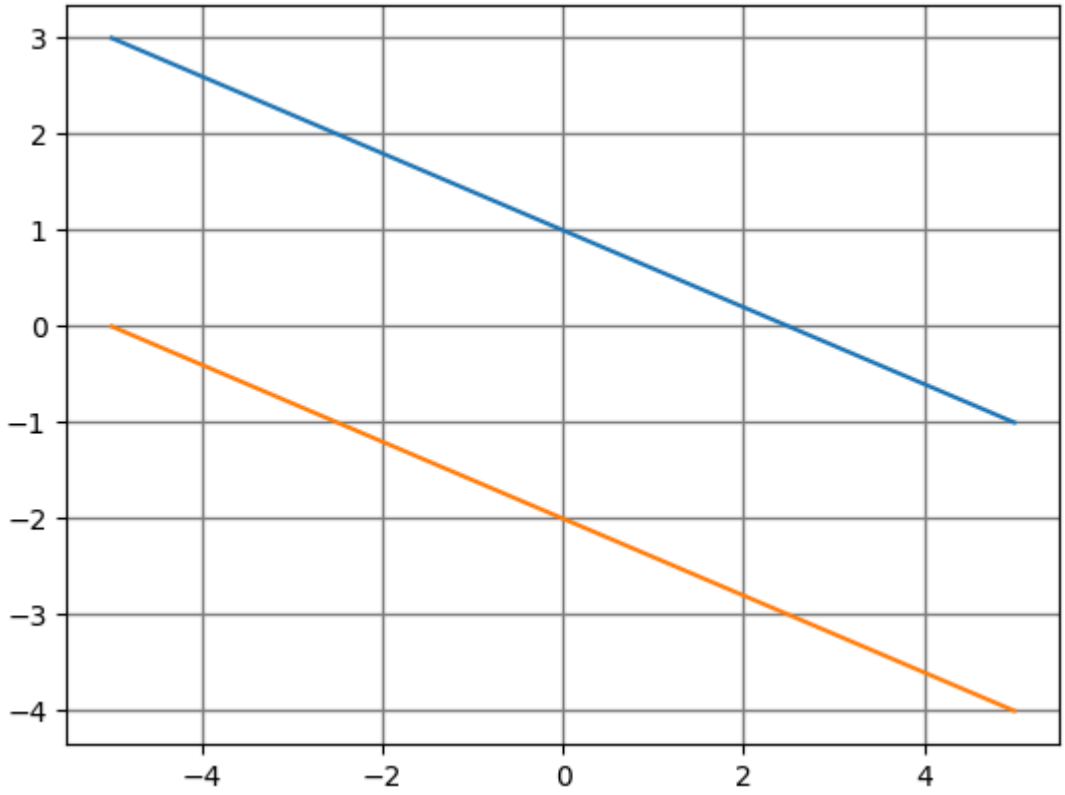
In [16]:
```
plot(lambda x: 3/4*x-1, lambda x: -x/2+4)
```



In [17]:
```
plot(lambda x: -4+3*x, lambda x: -4+3*x)
```



In [18]:
```
plot(lambda x: -2/5*x+1, lambda x: -2/5*x - 2)
```

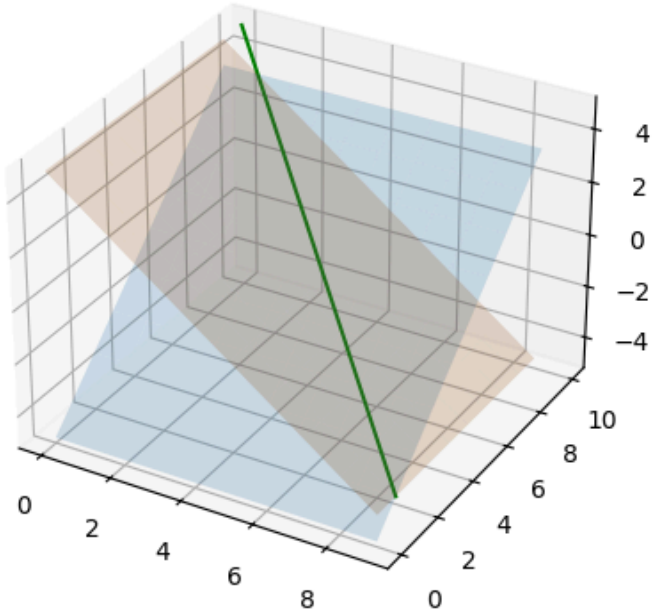


In [19]:
```
%matplotlib widget
xx, yy = np.meshgrid(range(10), range(10))
fig = plt.figure()
ax = fig.add_subplot(111,projection='3d')
ax.plot_surface(xx, yy, yy-5, alpha=0.2)
ax.plot_surface(xx, yy, -xx+5, alpha=0.2)
```

```
x = np.arange(10)
ax.plot(x , 10-x, 5 - x, color='green')
```

Out[19]: [<mpl_toolkits.mplot3d.art3d.Line3D at 0x1c37fd70050>]

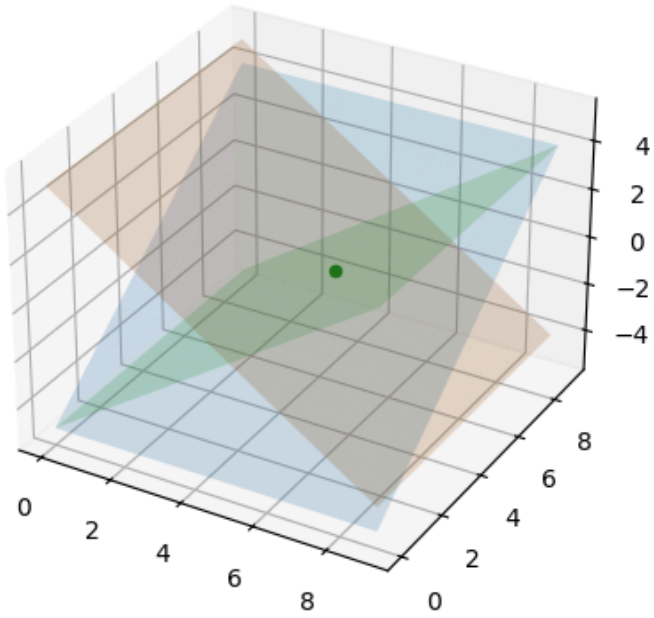
Figure



In [20]:
```
fig = plt.figure()
ax = fig.add_subplot(111,projection='3d')
ax.plot_surface(xx, yy, yy-5, alpha=0.2)
ax.plot_surface(xx, yy, -xx+5, alpha=0.2)
ax.plot_surface(xx, yy, xx-5, alpha=0.2)
ax.scatter(5 , 5, 0, color='green')
```

Out[20]: <mpl_toolkits.mplot3d.art3d.Path3DCollection at 0x1c37fe0f410>

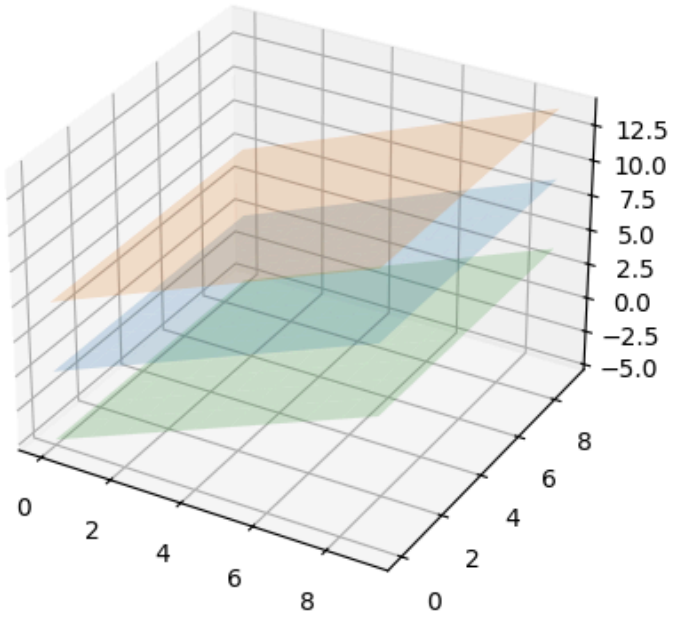
Figure



In [21]:
```
fig = plt.figure()
ax = fig.add_subplot(111,projection='3d')
ax.plot_surface(xx, yy, xx, alpha=0.2)
ax.plot_surface(xx, yy, xx+5, alpha=0.2)
ax.plot_surface(xx, yy, xx-5, alpha=0.2)
```

Out[21]:
```
<mpl_toolkits.mplot3d.art3d.Poly3DCollection at 0x1c31918d990>
```

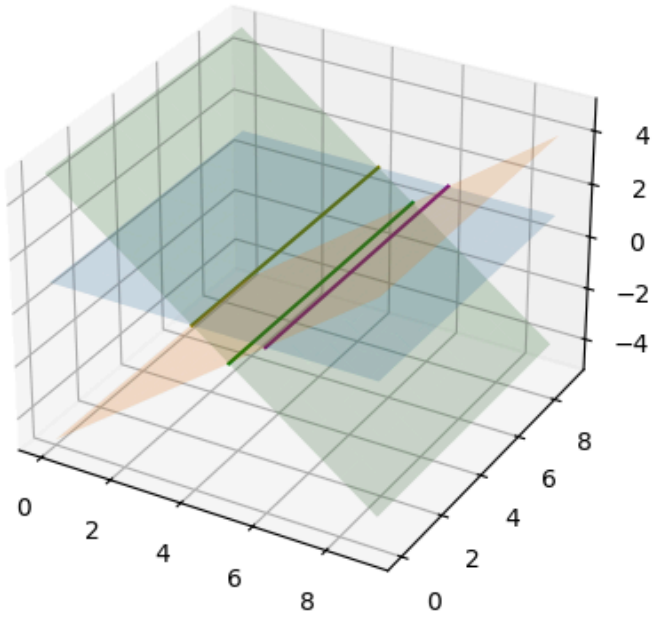
Figure



In [22]:
```
fig = plt.figure()
ax = fig.add_subplot(111,projection='3d')
ax.plot_surface(xx, yy, xx*0+1, alpha=0.2)
ax.plot_surface(xx, yy, xx-5, alpha=0.2)
ax.plot_surface(xx, yy, -xx+5, alpha=0.2)
x = np.arange(10)
ax.plot(0*x+5, x, 0*x, color='green')
ax.plot(0*x+4, x, 0*x+1, color='olive')
ax.plot(0*x+6, x, 0*x+1, color='purple')
```

Out[22]:
```
[<mpl_toolkits.mplot3d.art3d.Line3D at 0x1c319517a50>]
```

Figure



3.2 Lineare Ungleichungen

1

In [23]:
```
-1/4 < -1/5
```

Out[23]: True

In [24]:
```
(-4)**2 < (-5)**2 # 16 < 25
```

Out[24]: True

In [25]:
```
(-4)**3 > (-5)**3 # -64 > -125
```

Out[25]: True

In [26]:
```
-4**2 < (-5)**2 # -16 < 25
```

Out[26]: True

In [27]:
```
1/(-4)**2 > 1/(-5)**3 # 1/16 > 1/125
```

Out[27]: True

In [28]:
```
1/(-4)**3 < 1/(-(-5)**3) # -1/64 < 1/125
```

Out[28]: True

2

a)

a und b sind entweder beide größer als 0 oder beide kleiner als 0.

b)

In [29]:
```
%%latex
$-2c < 0 | :(-2)$ Division (oder Multiplikation) mit negativer Zahl dreht die Un

$c > 0$
```

$-2c < 0| : (-2)$ Division (oder Multiplikation) mit negativer Zahl dreht die Ungleichung um. $c > 0$

c)

Angenommen a ist 0. Dann ist keine der Ungleichungen erfüllt. a kann also nicht 0 sein.

Nun kann durch a^2 geteilt werden. Es ergibt sich $1 < a$ oder $1 > a$. a darf nicht 0 und nicht 1 sein.

In [30]:
```
import sympy
from sympy.abc import a, b, c, x, y
```

In [31]:
```
sympy.solve(a**2 < a**3) | sympy.solve(a**2 > a**3)
```

Out[31]: $(1 < a \wedge a < \infty) \vee (a > -\infty \wedge a < 1 \wedge a \neq 0)$

Als kleine Ergänzung noch die Antwort mittels der Python-Bibliothek sympy (symbolische Mathematik). Diese kommt auf das gleiche Ergebnis, hat allerdings auch das Verhalten im Unendlichen bedacht. Dieses hebt sich aber auf, da Unendlich oder negativ Unendlich je eine der Gleichung erfüllt.

3

a)

$-5x + 3 > -17| - 3$

$-5x > -20| : (-5)$

$x < 4$

$L_a = \{x|x \in \mathbb{R} \wedge x < 4\}$

b)

$0.6 - 3x < 3x - 2.4| - 0.6 - 3x$

$-6x < -3| : (-6)$

$x > \frac{1}{2}$

$L_b = \{x|x \in \mathbb{R} \wedge x > \frac{1}{2}\}$

c)

$-\frac{5}{9}x - 1 > -\frac{2}{3}| + 1$

$-\frac{5}{9}x > \frac{1}{3}| : (-\frac{5}{9})$

$x < -\frac{3}{5}| : (-\frac{5}{9})$

$L_c = \{x|x \in \mathbb{R} \wedge x < -\frac{3}{5}\}$

4

a)

$30 - x > 12| + x - 12$

$18 > x$

$L_a = \{x|x \in \mathbb{R} \wedge x < 18\}$

b)

$12 + x < 3| - 12$

$x < -9$

$L_b = \{x|x \in \mathbb{R} \wedge x < -9\}$

5

Gegeben ein Rechteck mit den Seiten a und b.

I $U = 2 * (a + b) > 20$

II $a = b + 2$

Einsetzen von II in I, sodass nur noch die Kürzere Seite b in der Ungleichung vorkommt.

$2 * ((b + 2) + b) > 20$

$2 * (b + 2 + b) > 20$

$2 * (2b + 2) > 20$

$4b + 4 > 20| - 4$

$4b > 16| : 4$

$b > 4$

6

a)

$(12 + a)/5 > 4|5$

$12 + a > 20| - 12$

$a > 8$

$L_a = \{x|x \in \mathbb{R} \wedge x > 8\}$

b)

$(a - 11) * 7 < 42| : 7$

$a - 11 < 6| + 11$

$a < 17$

$L_b = \{x|x \in \mathbb{R} \wedge x < 17\}$

c)

$(17 - 2x) * 3 \leq 15| : 3$

$17 - 2x \leq 5| - 17$

$-2x \leq -12| : (-2)$

$x \geq 6$

$L_c = \{x|x \in \mathbb{R} \wedge x \geq 6\}$

8

a)

$7 - a(x - 2) > 4| - 4 + a(x - 2)$

$3 > a(x - 2)$

Falls $a = 0$ ist die Ungleichung immer erfüllt ($L = \{x|x \in \mathbb{R} \wedge a = 0\}$)

a ist positive:

$\frac{3}{a} > x - 2| + 2$

$L = \{x|x \in \mathbb{R} \wedge a > 0 \wedge x < \frac{3}{a} + 2\}$

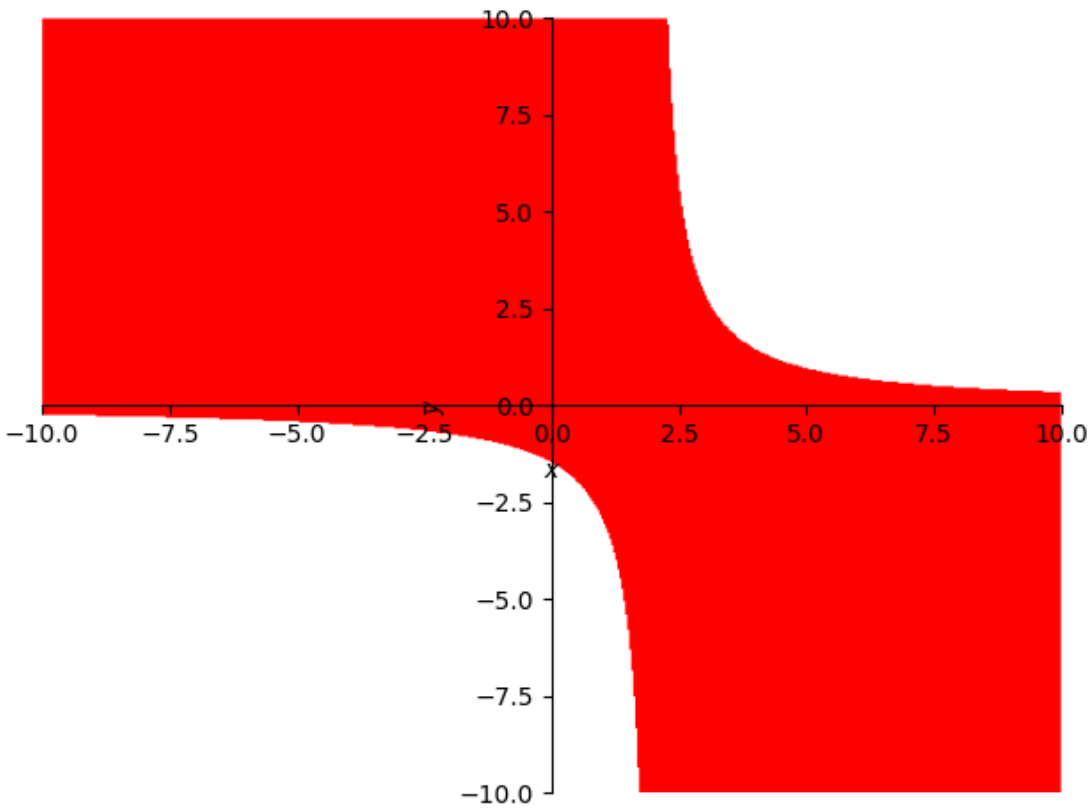
a ist negativ:

$\frac{3}{a} < x - 2| + 2$

$L = \{x|x \in \mathbb{R} \wedge a < 0 \wedge x > \frac{3}{a} + 2\}$

In [32]:
```
sympy.plot_implicit(7 - y * (x - 2) > 4, (x, -10, 10), (y, -10, 10), line_color=
```

Figure



Out[32]:
```
<sympy.plotting.plot.Plot at 0x1c31a9129d0>
```

b)

$(5 + q)(x - 11) > 2$

Fallunterscheidungen für $q < -5$, $q = -5$ und $q > -5$:

$q < -5 \Leftrightarrow 5 + q < 0$ also negativ.

$x - 11 < \frac{2}{5+q}| + 11$

$x < \frac{2}{5+q} + 11$

$L = \{x|x \in \mathrm{r} \wedge q < -5 \wedge x < \frac{2}{5+q} + 11\}$

$q = -5 \Leftrightarrow 5 + q = 0$

$0 * (x - 11) = 0 < 2$ Die Lösungsmenge ist leer.

$q > -5 \Leftrightarrow 5 + q > 0$

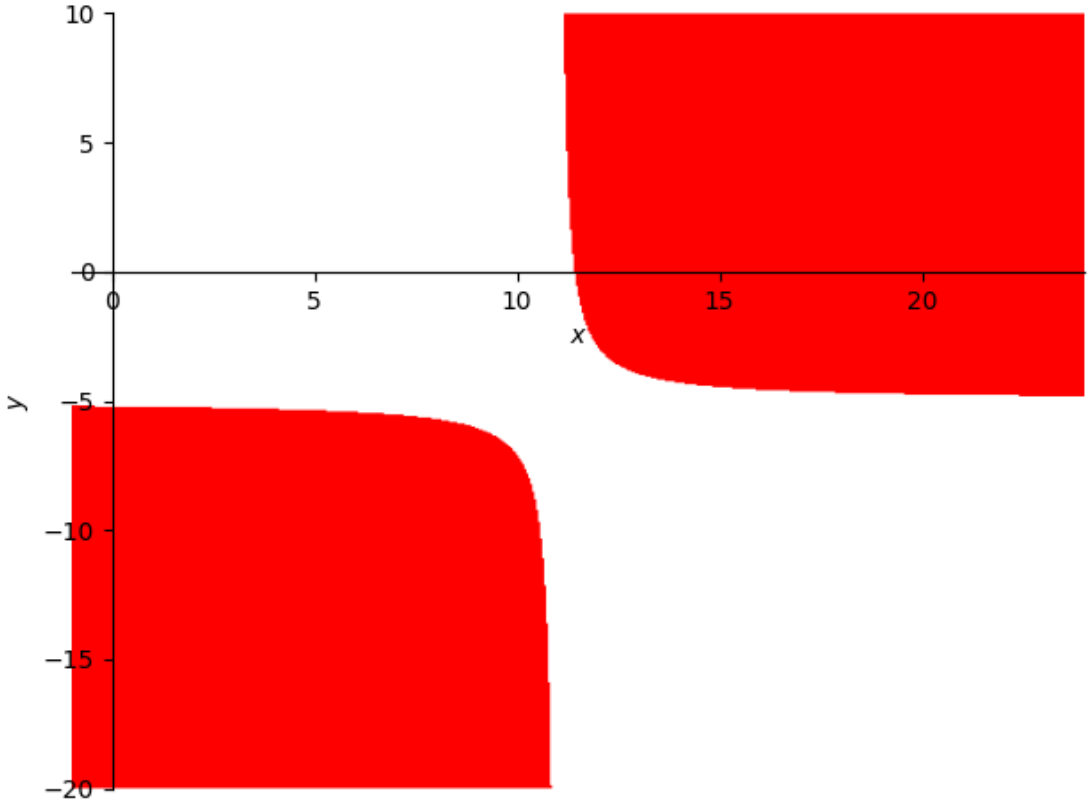
$x - 11 > \frac{2}{5+q} | + 11$

$x > \frac{2}{5+q} + 11$

$L = \{x | x \in \mathrm{r} \wedge q < -5 \wedge x > \frac{2}{5+q} + 11\}$

In [33]:
```
sympy.plot_implicit((5 + y) * (x - 11) > 2, (x, -1, 24), (y, -20, 10), line_colo
```

Figure



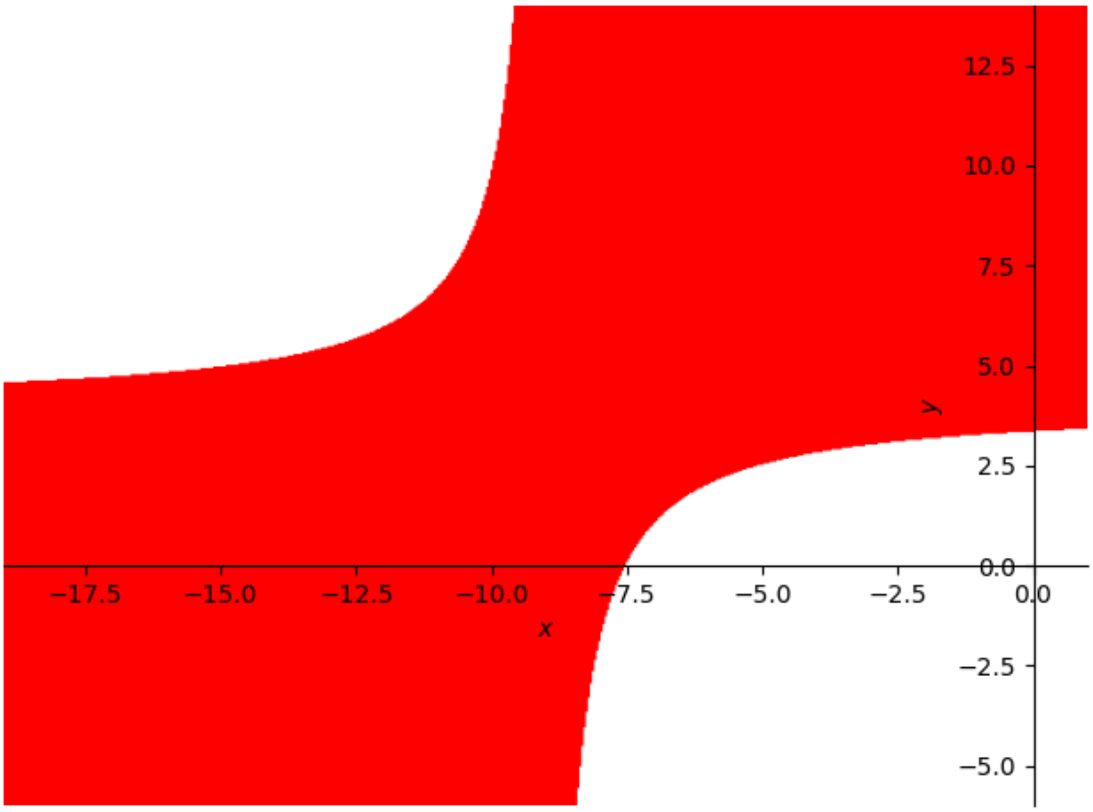
Out[33]:
```
<sympy.plotting.plot.Plot at 0x1c31a8e7150>
```

c)

Lösung ergibt sich aus der von d) mit $p = 4$:

In [34]:
```
sympy.plot_implicit((4 - y) * (x + 9) < 6, (x, -19, 1), (y, -6, 14), line_color=
```

Figure



Out[34]: <sympy.plotting.plot.Plot at 0x1c31aa40a10>

d)

$p = q$

$0 * (x + 9) = 0 < 6$

$L = \{x | x \in \mathbb{R} \wedge p = q\}$

Falls 2:

$p < q$

$x + 9 > \frac{6}{p-q} | - 9$

$x > \frac{6}{p-q} - 9$

$L = \{x | x \in \mathbb{R} \wedge p < q \wedge x > \frac{6}{p-q} - 9\}$

Falls 3:

$p > q$

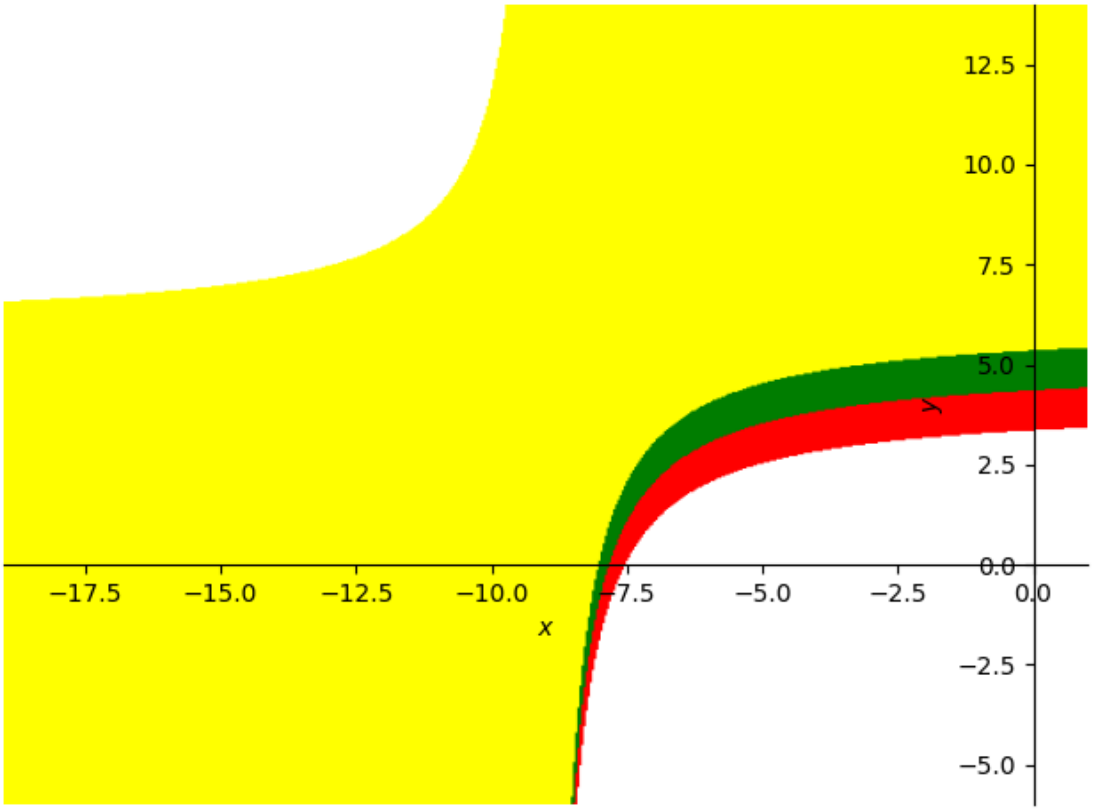
$x + 9 < \frac{6}{p-q} | - 9$

$x < \frac{6}{p-q} - 9$

$L = \{x | x \in \mathbb{R} \wedge p < q \wedge x < \frac{6}{p-q} - 9\}$

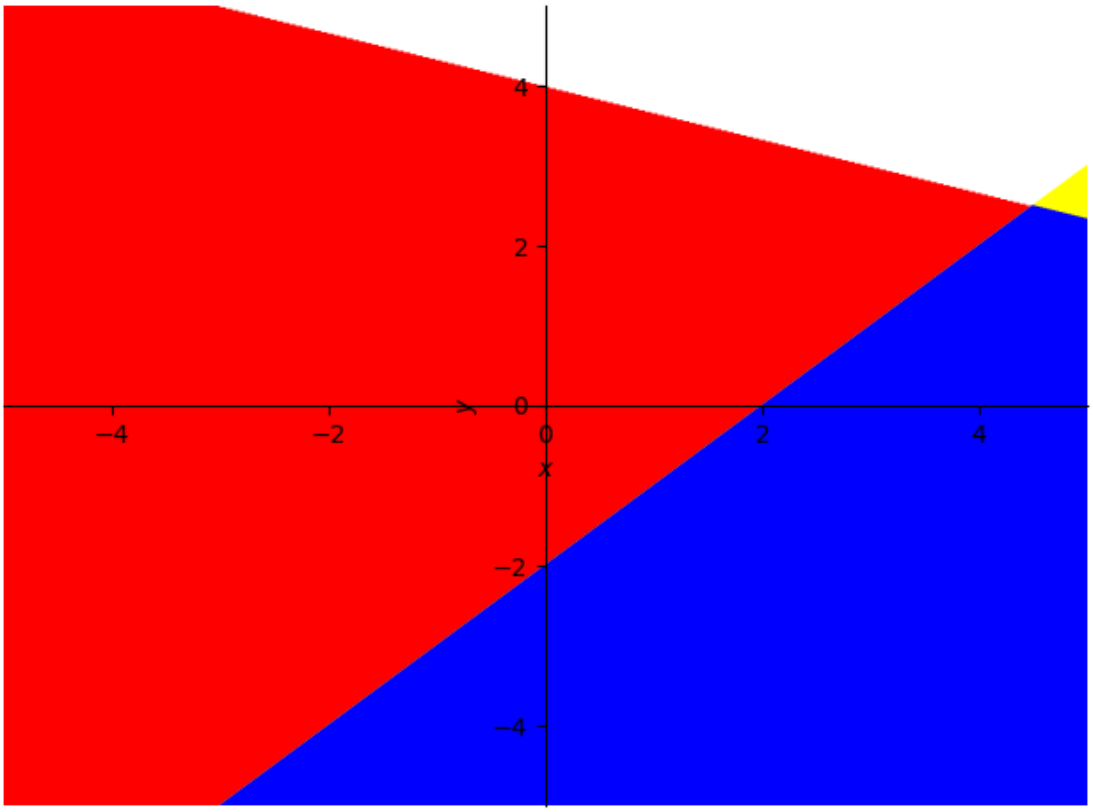
In [35]:
```
plot = sympy.plot_implicit((4 - y) * (x + 9) < 6, (x, -19, 1), (y, -6, 14), show
plot.append(sympy.plot_implicit((5 - y) * (x + 9) < 6, (x, -19, 1), (y, -6, 14),
plot.append(sympy.plot_implicit((6 - y) * (x + 9) < 6, (x, -19, 1), (y, -6, 14),
plot.show()
```

Figure



p wird veriiert (mangels 3d plot) und sorgt für eine vertikale Verschiebung.

9

In [36]:
```
f0 = x/3+y < 4
f1 = -x+y <= -2
plot = sympy.plot_implicit(f0, show=False, line_color='red')
plot.append(sympy.plot_implicit(f1, show=False, line_color='yellow')[0])
plot.append(sympy.plot_implicit(f0 & f1, show=False, line_color='blue')[0])
plot.show()
```

Figure